

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РФ

[Наименование образовательной организации]

[Наименование кафедры]

ОТЧЁТ

по лабораторной работе № 2

«Простейшие криптографические преобразования»

Выполнил: Сечейко Н.В.

Проверил(а): _____

Минск

2026

Содержание

1	Цель и содержание работы	2
2	Теоретические сведения	2
3	Исходные данные и вариант	3
4	Описание разработанной программы	4
5	Ход выполнения и результаты	4
6	Оценка криптографической стойкости	5
7	Алгоритм усложнения шифра	6
8	Заключение	7
A	Исходный код программы	7

1 Цель и содержание работы

Цель работы — изучить простейшие криптографические преобразования и разработать программу для шифрования, расшифрования и атаки шифра Виженера.

В соответствии с методическими указаниями программа должна:

1. реализовывать шифрование и расшифрование текста;
2. реализовывать атаку полным перебором ключа;
3. позволять оценить криптографическую стойкость шифра;
4. содержать алгоритм возможного усложнения шифра.

2 Теоретические сведения

Шифр Виженера является полиалфавитным шифром подстановки. В отличие от шифра Цезаря, в нём используется несколько алфавитов подстановки, которые циклически выбираются в соответствии с символами ключевого слова.

Каждому символу алфавита ставится в соответствие число от 0 до $n - 1$, где n — мощность алфавита. В работе используется английский алфавит в нижнем регистре:

abcdefghijklmnopqrstuvwxyz.

Мощность алфавита равна:

$$n = 26.$$

Пусть x_i — числовое значение символа открытого текста, k_i — числовое значение соответствующего символа циклически повторяемого ключа, а y_i — числовое значение символа шифртекста. Тогда шифрование выполняется по формуле:

$$y_i = (x_i + k_i) \bmod n.$$

Расшифрование выполняется по формуле:

$$x_i = (y_i - k_i) \bmod n.$$

Если длина ключа меньше длины сообщения, ключ повторяется. Например, для текста **attackatdawn** и ключа **dog** используется последовательность:

dogdogdogdog.

Основная уязвимость шифра Виженера заключается в циклическом повторении ключа. При известной длине ключа возможен полный перебор всех вариантов ключа. Если длина ключа равна m , количество ключей составляет:

$$N_{\text{ключей}} = n^m.$$

Среднее число попыток при полном переборе:

$$N_{\text{ср}} = \frac{n^m}{2}.$$

3 Исходные данные и вариант

Вариант 26 — шифр Виженера.

В качестве языка исходного текста выбран английский язык. Программа работает со строчными буквами английского алфавита без пробелов и знаков препинания.

4 Описание разработанной программы

Программа написана на языке C++. После запуска пользователю доступно меню:

1. шифрование открытого текста;
2. расшифрование шифртекста;
3. атака полным перебором ключа;
4. завершение работы.

При шифровании программа получает открытый текст и ключ. Для каждого символа вычисляется сумма его номера и номера соответствующего символа ключа по модулю 26.

При расшифровании программа получает шифртекст и ключ. Из номера символа шифртекста вычитается номер символа ключа по модулю 26.

Для атаки полным перебором программа получает шифртекст, известный открытый текст и длину ключа. Затем она последовательно формирует все ключи заданной длины от символов **a** до **z**. Для каждого ключа выполняется расшифрование, после чего результат сравнивается с известным открытым текстом.

5 Ход выполнения и результаты

Для проверки программы использованы следующие исходные данные:

- открытый текст: `attackatdawn`;
- ключ: `dog`;
- длина ключа: $m = 3$.

Результат шифрования:

`attackatdawn` $\longrightarrow$ `dhzdqqdhjdkt`.

Проверка расшифрования:

`dhzdqqdhjdkt` $\longrightarrow$ `attackatdawn`.

При атаке полным перебором в качестве исходного открытого текста указано `attackatdawn`. Программа находит ключ `dog`. Полное количество проверяемых ключей равно:

$$26^3 = 17576.$$

Среднее количество попыток:

$$\frac{26^3}{2} = 8788.$$

Ключ `dog` находится в переборе после 2399 проверок, если ключи перебирать в лексикографическом порядке. В программе после обнаружения совпадения перебор продолжается до проверки всех ключей заданной длины.

6 Оценка криптографической стойкости

Стойкость реализованного шифра зависит от длины ключа. Для английского алфавита количество возможных ключей определяется выражением 26^m .

Для ключа длиной 1:

$$26^1 = 26.$$

Для ключа длиной 3:

$$26^3 = 17576.$$

Для ключа длиной 5:

$$26^5 = 11881376.$$

Таким образом, короткий ключ шифра Виженера может быть найден полным перебором. Дополнительную информацию атакующему даёт повторение ключа в шифртексте. Для более длинных сообщений возможно определить длину ключа методом прореживания и затем провести криптоанализ совокупности шифров Цезаря.

Реализованный шифр не предназначен для защиты информации в современных информационных системах. Его криптографическая стойкость существенно ниже стойкости современных симметричных алгоритмов.

7 Алгоритм усложнения шифра

Для усложнения базового шифра Виженера можно использовать ключ, длина которого равна длине сообщения. Алгоритм имеет следующий вид:

1. сгенерировать случайную последовательность символов длиной не меньше длины открытого текста;
2. использовать эту последовательность как ключ;
3. для каждого символа текста выполнить сложение по модулю 26;
4. передать получателю шифртекст и ключ по разным защищённым каналам;
5. после однократного использования уничтожить ключ.

При условии случайности ключа, равенства его длины длине сообщения и однократного использования такой подход соответствует идее одноразового шифроблокнота и существенно усложняет атаку по повторяющемуся ключу.

8 Заключение

В ходе лабораторной работы для варианта 26 реализован шифр Виженера на английском алфавите. Программа выполняет шифрование и расшифрование текста, а также атаку полным перебором ключа по известному открытому тексту.

На примере текста `attackatdawn` и ключа `dog` получен шифртекст:

`dhzdqqdhjdk.`

Полученный текст успешно расшифровывается исходным ключом. Для ключа длиной 3 существует 17576 вариантов, поэтому такой ключ может быть найден полным перебором.

A Исходный код программы

```
#include <cstdint>
#include <iostream>
#include <limits>
#include <string>

using namespace std;

namespace {

const string kAlphabet = "abcdefghijklmnopqrstuvwxyz";

bool isAlphabetSymbol(char symbol) {
    return symbol >= 'a' && symbol <= 'z';
}

bool isValidText(const string& text) {
    if (text.empty()) {
```

```

        return false;
    }

    for (char symbol : text) {
        if (!isAlphabetSymbol(symbol)) {
            return false;
        }
    }
    return true;
}

bool isValidKey(const string& key) {
    return isValidText(key);
}

string readText(const string& prompt) {
    string value;
    while (true) {
        cout << prompt;
        getline(cin, value);
        if (!cin) {
            return {};
        }
        if (isValidText(value)) {
            return value;
        }
        cout << "Используйте только строчные буквы английского  

        ↪ алфавита.\n";
    }
}

string readKey() {

```

```

string key;
while (true) {
    cout << "Введите ключ: ";
    getline(cin, key);
    if (!cin) {
        return {};
    }
    if (isValidKey(key)) {
        return key;
    }
    cout << "Ключ должен содержать только строчные буквы
    ↪ английского алфавита.\n";
}
}

size_t readKeyLength() {
    size_t length = 0;
    while (true) {
        cout << "Введите длину ключа: ";
        if (cin >> length && length > 0) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return length;
        }
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Длина ключа должна быть положительным
        ↪ числом.\n";
    }
}

string transform(
    const string& text,

```

```

const string& key,
bool encrypt) {
string result = text;

for (size_t index = 0; index < text.size(); ++index) {
    const int textValue = text[index] - 'a';
    const int keyValue = key[index % key.size()] - 'a';
    const int value = encrypt
        ? (textValue + keyValue) %
          ↪ static_cast<int>(kAlphabet.size())
        : (textValue - keyValue +
          ↪ static_cast<int>(kAlphabet.size())) %
          static_cast<int>(kAlphabet.size());
    result[index] = kAlphabet[static_cast<size_t>(value)];
}

return result;
}

void searchKey(
    const string& encryptedText,
    const string& originalText,
    string& candidate,
    size_t position,
    size_t& attempts,
    bool& found) {
    if (position == candidate.size()) {
        ++attempts;
        if (transform(encryptedText, candidate, false) ==
            ↪ originalText) {
            cout << "Найден ключ: " << candidate << "\n";
            found = true;
        }
    }
}

```

```

        }
        return;
    }

    for (char symbol : kAlphabet) {
        candidate[position] = symbol;
        searchKey(
            encryptedText,
            originalText,
            candidate,
            position + 1,
            attempts,
            found);
    }
}

void encryptText() {
    const string text = readText("Введите открытый текст: ");
    if (text.empty()) {
        return;
    }
    const string key = readKey();
    if (key.empty()) {
        return;
    }
    cout << "Зашифрованный текст: " << transform(text, key,
        ↪ true) << "\n";
}

void decryptText() {
    const string text = readText("Введите зашифрованный текст:
    ↪ ");

```

```

    if (text.empty()) {
        return;
    }
    const string key = readKey();
    if (key.empty()) {
        return;
    }
    cout << "Расшифрованный текст: " << transform(text, key,
        ↪ false) << "\n";
}

```

```

void bruteForceAttack() {
    const string encryptedText =
        readText("Введите зашифрованный текст: ");
    if (encryptedText.empty()) {
        return;
    }

    const string originalText =
        readText("Введите исходный открытый текст: ");
    if (originalText.empty()) {
        return;
    }

    if (encryptedText.size() != originalText.size()) {
        cout << "Длины текстов должны совпадать.\n";
        return;
    }

    const size_t keyLength = readKeyLength();
    string candidate(keyLength, kAlphabet[0]);
    size_t attempts = 0;
}

```

```

bool found = false;

searchKey(
    encryptedText,
    originalText,
    candidate,
    0,
    attempts,
    found);

cout << "Проверено ключей: " << attempts << "\n";
if (!found) {
    cout << "Подходящий ключ не найден.\n";
}
}

int readAction() {
    int action = 0;
    while (true) {
        cout << "\n1. Зашифровать текст\n"
              << "2. Расшифровать текст\n"
              << "3. Выполнить атаку полным перебором ключа\n"
              << "4. Выход\n"
              << "Выберите действие: ";
        if (cin >> action &&
            action >= 1 &&
            action <= 4) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return action;
        }
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

```

```

        cout << "Введите число от 1 до 4.\n";
    }
}

}

int main() {
    cout << "Лабораторная работа №2. Шифр Виженера\n"
        << "Вариант 26. Вариант 2 из 4.\n"
        << "Используется английский алфавит.\n";

    while (true) {
        switch (readAction()) {
            case 1:
                encryptText();
                break;
            case 2:
                decryptText();
                break;
            case 3:
                bruteForceAttack();
                break;
            case 4:
                return 0;
        }
    }
}

```